

2025单元测试二（图与排序）

题量： 11 满分： 100.0

 智能分析

100.0 分

考试时间： 2025-05-11 19:00 至 2025-05-11 20:00

一. 单选题（共 4 题， 20.0 分）

1. (单选题, 5.0 分)

适用于顺序查找的表的存储方式及元素排列要求为（ ）。

- A.
顺序存储结构，元素有序
- B.
链式存储结构，元素有序
- C.
顺序或链式结构存储，元素有序
- D.
顺序或链式结构存储，元素不要求有序

我的答案:D



5.0 分

2. (单选题, 5.0 分)

设有1024个元素的有序表，采用折半查找方法，查找成功时最多的比较次数是。

- A.
1024
- B.
512
- C.
10
- D.
11

我的答案:D



5.0 分

3. (单选题, 5.0 分)

在下列排序方法中，算法的平均复杂度不为 $O(n \log n)$ 的是（ ）。

1.

- A.
堆排序
- B.
简单选择排序
- C.
快速排序
- D.
归并排序

我的答案:B



5.0 分

4. (单选题, 5.0 分)

设无向图的顶点个数为 n , 则该图最多有 () 条边

- A.
 $n(n-1)$
- B.
 $n(n-1)/2$
- C.
 $n(n+1)/2$
- D.
 n^2

我的答案:B



5.0 分

二. 填空题 (共 2 题, 10.0 分)

5. (填空题, 5.0 分)

若无向图 $G=(V, E)$ 中含7个顶点, 则保证图 G 在任何情况下都是连通的, 则需要的边数最少是()。

我的答案:

(1) 16



5.0 分

6. (填空题, 5.0 分)

对数据序列 (288,271,260,331,217,215,56,199,28,23) , 采用最低位优先的基数排序进行升序排序, 第1趟后的排序结果为 () (注: 序列中数字以英文的逗号分隔,直接写数字序列, 不再写括号,数字不足3位的前面补0, 如: 23写为023) 。

我的答案:

(1) 260,271,331,023,215,056,217,288,028,199



5.0 分

三. 判断题 (共 2 题, 10.0 分)

7. (判断题, 5.0 分)

若无向图的邻接矩阵中主对角线上的元素全是 0, 其余元素全是 1, 则可以断定该图一定是无向完全图。

A. 对

B. 错

我的答案:对

5.0 分

8. (判断题, 5.0 分)

对少量数据或基本有序的数据进行排序时, 直接插入排序的时间开销可能低于快速排序。

A. 对

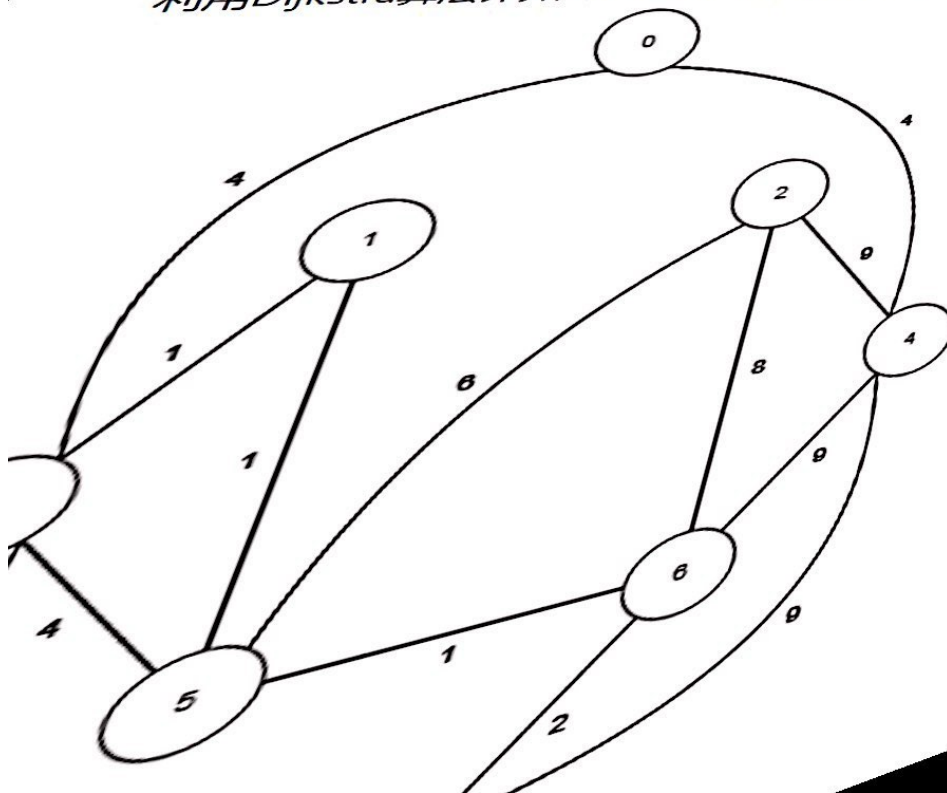
B. 错

我的答案:对

5.0 分

9. (简答题, 15.0 分)

利用Dijkstra算法计算图中从0到2的最短路径及长度 (需给出中间计算过程)



9. 从 1 开始 dist 为路径长度, final 为是否访问完成. 1 为初始点, 1 为终点

	dist	final	parent		dist	final	parent
0	0	T		0	0	T	3→1
1	∞	F	-1	1	5	F	4→2
2	∞	F	-1	2	13	F	0→3
3	4	F	1→3	3	4	T	0→3
4	4	F	1→4	4	4	T	3→5
5	∞	F	-1	5	8	F	4→6
6	∞	F	-1	6	13	F	3→7
7	∞	T	-1	7	5	F	

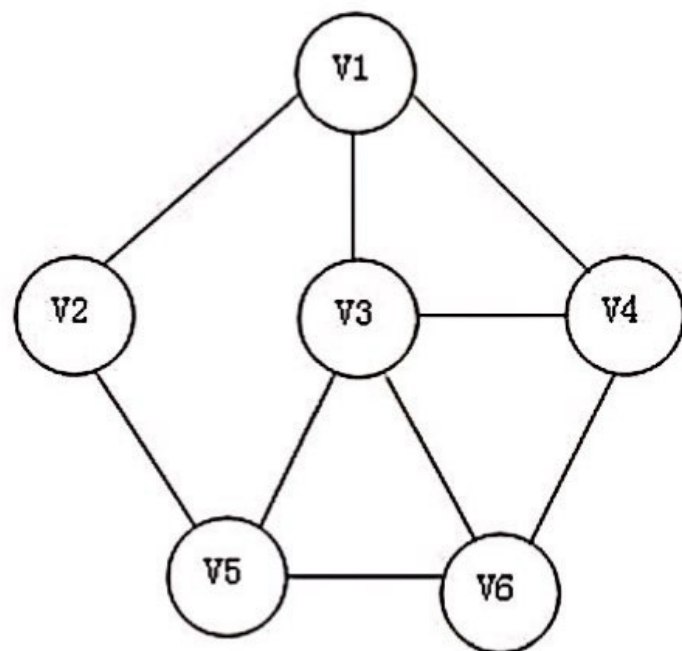
	dist	final	parent		dist	final	parent
0	0	T		0	0	T	3→1
1	5	T	3→1	1	5	T	5→2
2	13	F	4→2	2	12	F	0→3
3	4	T	0→3	3	4	T	0→4
4	4	T	0→4	4	4	T	1→5
5	6	F	1→5	5	6	T	7→6
6	7	F	7→6	6	7	F	3→7
7	5	T	3→7	7	5	T	

	dist	final	parent		dist	final	parent
0	0	T		0	0	T	3→1
1	5	T	3→1	1	5	T	5→2
2	12	F	5→2	2	12	T	0→3
3	4	T	0→3	3	4	T	0→4
4	4	T	0→4	4	4	T	1→5
5	6	T	1→5	5	6	T	7→6
6	7	T	7→6	6	7	T	3→7
7	5	T	3→7	7	5	T	

故 0 到 2 最短路径为 0→3→1→5→2
长度为 12

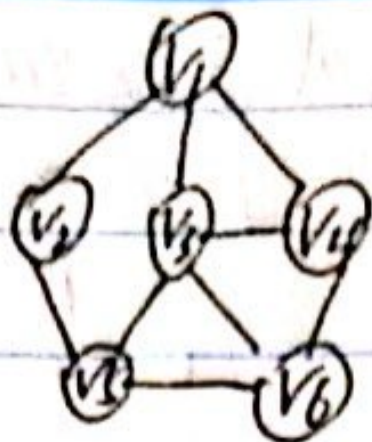
10. (简答题, 15.0 分)

已知下图



- (1) 试写出从顶点V1出发的BFS遍历序列;
- (2) 画出从顶点V1出发的BFS树。

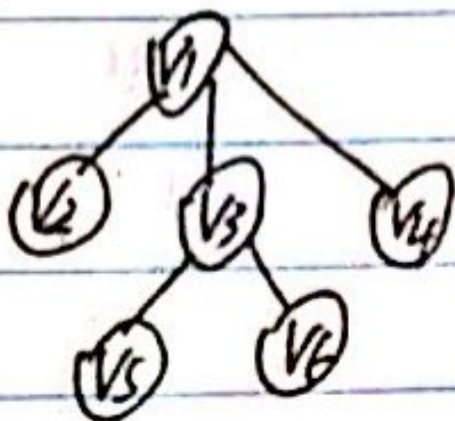
10.



(1) 从 V_1 出发 BFS 序列

$V_1 \rightarrow V_2 \rightarrow V_3 \rightarrow V_4 \rightarrow V_5 \rightarrow V_6$

(2) BFS 树



11. (算法设计, 30.0 分)

【算法设计题】设计一个算法确定一个无向图是否包含回路。

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用C++语言描述算法, 关键之处给出注释。
- (3) 分析算法的时间复杂度

要求: 在纸上手写并签名, 拍照上交。

照片要清晰, 大小合适。

我的答案:

30 分

(1) 算法思想

使用DFS判断无向图中是否存在回路，DFS通过递归遍历图中的节点，同时记录当前节点是否在递归栈中，如果在DFS遍历过程中，某节点已被访问并且仍在递归栈中，则说明图中包含回路。

(2) C++代码

DFS辅助函数，用于检查是否存在回路
bool dfs(int node, vector<vector<int>>& graph, vector<bool>& visited, vector<bool>& inStack)

```
{
    visited[node] = true;
    inStack[node] = true;
    // 遍历当前结点的邻居
    for (int neighbor : graph[node])
    {
        if (!visited[neighbor] && dfs(neighbor, graph, visited, inStack))
        {
            // 如果邻居结点未被访问则递归访问
            return true; // 若检测到回路返回true
        }
    }
}
```

else if (inStack[neighbor])

```
{
    return true; // 若邻居结点已在递归栈中，则存在回路
}
```

```
}
inStack[node] = false; // 递归完成当前结点后，从递归栈中移除
return false; // 未检测到回路
```

// 检测图中是否有回路主函数

bool containsCycle (int n, vector<vector<int>> & graph)

```
{
    vector<bool> visited (n, false);
    vector<bool> inStack (n, false);
    for (int i = 0; i < n; i++) // 遍历每个节点
    {
        if (!visited[i] && dfs(i, graph, visited, inStack))
            return true;
    }
    return false;
}
```

3) 时间复杂度: $O(V+E)$

每个节点与每条边各访问一次，故时间复杂度 $O(V+E)$

V为结点数，E为边数。